

Building a Python Adventure Game with GitHub Copilot

Course-End Project Report — Legend of the Lost Treasure

1. Project Overview

This project implements a text-based adventure game, **adventure_game.py**, built in Python and developed with the assistance of GitHub Copilot in VS Code. The player takes on the role of an explorer searching for a legendary treasure. After entering their name, the player chooses between two paths — a dense forest or a mysterious cave — and each path presents a further decision that determines whether the quest ends in success or failure. The game loops so the player can restart after any attempt.

2. How GitHub Copilot Assisted in Writing and Optimizing the Code

GitHub Copilot was used throughout development to speed up and refine the code:

- **Function scaffolding:** Once a docstring or comment describing the intended behavior of `start_game()`, `forest_path()`, and `cave_path()` was written, Copilot suggested complete function bodies, including narrative text and input-handling logic, which were then reviewed and edited for tone and correctness.
- **Input validation:** Copilot proposed the `get_choice()` helper pattern to repeatedly prompt the player until a valid option was entered, reducing duplicated validation code across the forest and cave scenarios.
- **Conditional logic:** For the if-else structures handling player choices, Copilot suggested the branching outcomes (win vs. lose) based on the narrative context established in surrounding comments.
- **Code efficiency:** Copilot recommended consolidating repeated print/input patterns into reusable functions (`slow_print()` and `get_choice()`) instead of writing the same logic separately in each path function, which reduced overall code duplication.
- **Game loop:** Copilot helped draft the `main()` loop that re-calls `start_game()` and asks the player whether to restart, matching the 'restart' ending requirement.

3. Key Challenges Faced

- **Balancing narrative detail with concise code:** Copilot's initial suggestions for descriptive text were sometimes verbose; these were trimmed to keep the game readable and quick to play.
- **Input validation edge cases:** Handling blank input for the player's name and unexpected text at choice prompts required adding explicit checks and a reusable validation loop rather than relying solely on generated code.

- Structuring outcomes clearly: Ensuring every path led unambiguously to a 'win' or 'lose' result (rather than leaving branches unresolved) required manually tracing each conditional path after accepting Copilot's suggestions.
- Keeping the restart loop clean: The first draft of the main loop risked infinite recursion by calling `start_game()` from within itself; this was refactored into a while loop in `main()` driven by a simple yes/no prompt.

4. Enhancements and Modifications to the Original Structure

- Added a `slow_print()` helper that prints text character-by-character to give the game a more immersive, old-school adventure feel.
- Added a `get_choice()` helper that validates all player input against a list of accepted answers, so the game never crashes or stalls on unexpected input.
- Split the ending logic into a dedicated `show_result()` function so win/lose messaging is handled consistently regardless of which path the player took.
- Wrapped the entire game in a `main()` loop with a yes/no restart prompt, fulfilling the 'restart after an unsuccessful attempt' requirement while also allowing a player to replay after winning.
- Added a default player name ('Explorer') as a fallback if the player presses enter without typing a name, preventing empty-string bugs later in the game.

5. Conclusion

The final `adventure_game.py` script meets all project requirements: it introduces the quest, collects the player's name, offers a branching choice between a forest and a cave, resolves each path through conditional logic into a win or lose outcome, and loops to allow restarting. GitHub Copilot meaningfully accelerated development by generating first drafts of functions and boilerplate patterns, while manual review remained essential for correctness, tone, and avoiding structural issues such as unintended recursion.